Prostate Cancer Dashboard — AI Pipeline · NLP Classifier DB Tables

> Which table gets filled with what, step by step. Current schema. Korean mirror: `AI_PIPELINE_NLP_DB_TABLES_KR.md`. If they diverge, `models.py` wins.

1. Overall data flow — step → table

Input transcript → **NLP 7 steps** (preprocess · segment · 5-model classify · top-N · context · export · persist) → **AI 5 steps** (score · extract · filter · select · reformat) → DB → dashboard.

```
input xlsx transcript
   |
   ▼ NLP 7  (sentence_classification + nlp-classifiers Docker)
   1 preprocess(doctor rows only)  2 segment(stringi)  3 classify ×5 RF(prob)
   4 top-N select  5 context(<main>)  6 export xlsx  7 persist
   |   → transcript_analysis_log(run header) · nlp_all_predictions · sentence_prediction
   |     · nlp_pipeline_intermediate(step snapshots) · patient_summary(parent key)
   ▼ AI 5  (ai_pipeline, Azure GPT-4o)
   1 score 0-5  2 extract(estimate+treatment)  3 filter  4 select 1  5 reformat(patient sentence)
   |   → llm_pipeline_intermediate(intermediate candidates) · llm_domain_scoring_and_summary(final, patient-facing)
   ▼
dashboard (doctor/patient)
```

2. Step → table fill matrix

Pipeline step	Table(s) filled	What / how	Example rows (SID14)
NLP 1 preprocess	(none)	doctor-rows-only in-memory filter	—
NLP 2 segment	`transcript_analysis_log` · `nlp_pipeline_intermediate`	run header (`total_sentences`) · step snapshot (`step='segmentation'` , `payload` JSONB)	1 · 424 sentences
NLP 3 classify ×5	`nlp_all_predictions`	every sentence × 5-model probs (`pred_cp/le/ed/inc/ius`) 1 row/sentence	~424 rows
NLP 4 top-N + 5 context	`sentence_prediction`	per-domain top-N + context (`<main>`). 1 row/sentence·domain	50 rows (5×10)
NLP 6 export / 7 persist	`transcript_analysis_log` · `patient_summary`	store xlsx (`xlsx_data`) · create patient **parent row**	1 + 1
AI 1 score · 2 extract · 3 filter · 4 select	`llm_pipeline_intermediate`	per-candidate intermediate: `ai_score` · `estimate` · `treatment` · `survived_filter`	50 rows
AI 4 select + 5 reformat	`llm_domain_scoring_and_summary`	final domain output (patient-facing): `source_sentence` · `source_context` (`<main>`) · `reformat_sentence` · `ai_score` · `treatment`	~6 (treatment branches)
AI end	`transcript_analysis_log` UPDATE	`ai_overall_score` , `processed=true`	—

> **Patient input is NOT the pipeline** : first-visit Risk answers + follow-up surveys are stored to `patient_survey_submission_log` when the patient enters them on screen (not the pipeline).

3. ERD — table relationships

- **`transcript_analysis_log` (run header)** is the 1:N parent of the NLP·AI result tables (FK `analysis_id`).
- **`patient_summary` (file+speaker)** is the parent anchor of the patient-side child `patient_survey_submission_log` .

```
transcript_analysis_log (id PK, patient_id, source_filename, total_sentences, top_n, ai_overall_score, processed, xlsx_data)
   │ analysis_id (FK)
   ├─ nlp_all_predictions        (id, analysis_id, sentence_index, sentence_text, pred_cp/le/ed/inc/ius, context)
   ├─ sentence_prediction        (id, analysis_id, model[cp/le/ed/inc/ius], sentence_index, pred_score, sentence_text, context<main>)
   ├─ nlp_pipeline_intermediate  (id, analysis_id, step, payload jsonb, row_count)
   ├─ llm_pipeline_intermediate  (id, analysis_id, domain, ai_score, estimate, treatment, survived_filter, score_explanation)
   └─ llm_domain_scoring_and_summary (id, analysis_id, domain, ai_score, treatment, source_sentence, source_context, reformat_sentence)

patient_summary (file PK, speaker PK)
   └─ patient_survey_submission_log (id, file+speaker FK, survey_type, answers jsonb, redcap_*)   ← patient survey answers (first-visit Risk = risk_perception_2, follow-up = sdm/dcs/satisfaction)
```

4. Per-table detail

- **`transcript_analysis_log`** (run header): one analysis = one row. Parent of all result tables (FK `analysis_id`). Filled: NLP 2 (create) → 7, AI end (UPDATE).
- **`nlp_all_predictions`** (NLP): all sentences × 5 RF-model probabilities, fully retained. 1 row/sentence. Audit/analysis.
- **`sentence_prediction`** (NLP): per-domain top-N sentences + context (`<main>…</main>`). Read by: doctor view · admin.
- **`nlp_pipeline_intermediate`** (NLP): per-step JSONB snapshots. Debug/audit.
- **`llm_pipeline_intermediate`** (AI): per-candidate intermediate (score/extract/filter/select).
- **`llm_domain_scoring_and_summary`** (AI final): patient-facing final output. Read by: `/api/patient/ai-summary`.
- **`patient_summary`** (patient parent): file+speaker parent anchor.

Source code: NLP = `sentence_classification/` + nlp-classifiers Docker · AI = `ai_pipeline/` (Azure GPT-4o) · persistence = `db/persistence_helper.py` + `app/Backend/persistence.py` (`save_all` : the 5 NLP-side tables).